UNIT-2

REACT BASICS

2.1 DIFFERENT COMPONENTS

2.1.1 Class Component

Before React 16.8, Class components were the only way to track the state and lifecycle of a React component. Function components were considered "stateless".

With the addition of Hooks, Function components are now almost equivalent to Class components. The differences are so minor that you will probably never need to use a Class component in React.

Even though Function components are preferred, there are no current plans to remove Class components from React.

Example

Header.js

```
import React from "react";
class Header extends React.Component {
render()
{
      return(
            <div>
                  <h1>Diploma Computer /It Engineering </h1>;
            </div>
      );
  }
}
export default Header;
```

App.js

```
import './App.css';
import Header from './Header';

function App() {
  return (
    <div className="App">
      <Header></Header>
    </div>
  );
}

export default App;
```

Output:

Diploma Computer/It Engineering.

2.1.2 Function component

A **React functional component** is a straight JavaScript function that takes props and returns a React element. Writing functional components has been the traditional way of writing React components in advanced applications since the advent of **React Hooks**.

A React component's primary function is to classify the displayed view and connect it to the code that governs its actions. **React's functional components** reduce this to the most basic profile feasible: a function that accepts properties and returns a JSX definition. The function body contains all of the definitions needed for actions, and the class-related sections of object-oriented components are omitted.

Example

Header.js

```
import React from 'react'
import './App.css';
function Header()
{
return (
      <h1>My First Functional Component</h1>
);
}
export default Header;
```

App.js

```
import './App.css';
import Header from './Header';

function App() {
  return (
    <div className="App">
      <Header></Header>
    </div>
  );
}

export default App;
```

Output:

My First Functional Component

2.2 PROPS

Props stand for "**Properties**." They are **read-only** components. It is an object that stores the value of attributes of a tag and works similarly to the HTML attributes. It gives a way to pass data from one component to other components. It is similar to function arguments. Props are passed to the component in the same way as arguments passed in a function.

Props are **immutable** so we cannot modify the props from inside the component. Inside the components, we can add attributes called props. These attributes are available in the component as **this.props** and can be used to render dynamic data in our render method.

When you need immutable data in the component, you have to add props to **reactDom.render()** method in the **main.js** file of your ReactJS project and use it inside the component that you need. It can be explained in the below example.

Example

Create a variable named carName and send it to the Cars component:

Cars.js

```
import React from 'react';

function Car(props) {
  return <h2>I am an {props.brand}!</h2>;
}
function Cars() {
  const carName = "Audi";
  return (
    <>
     <h1>Describe how the vehicle looks and runs</h1>
     <Car brand={carName} />
    </>
  );
}
export default Cars;
```

App.js

```
import './App.css';
import Cars from './Cars';

function App() {
  return (
    <div className="App">
      <Cars></Cars>
    </div>
```

```
  );
}

export default App;
```

Output:

Describe how the vehicle looks and runs

I am an Audi!

2.3 STATE

The state is an updatable structure that is used to contain data or information about the component. The state of a component can change over time. The change in state over time can happen as a response to user action or system event. A component with a state is known as a stateful component. It is the heart of the react component that determines the behaviour of the component and how it will render. They are also responsible for making a component dynamic and interactive.

A state must be kept as simple as possible. It can be set by using the **setState()** method and calling setState() method triggers UI updates. A state represents the component's local state or information. It can only be accessed or modified inside the component or by the component directly. To set an initial state before any interaction occurs, we need to use the **getInitialState()** method.

Defining the state

To define a state, you have to first declare a default set of values for defining the component's initial state. To do this, add a class constructor which assigns an initial state using this.state. The '**this.state**' property can be rendered inside **render()** method.

Example

Cars.js

```
import React from 'react';
class Cars extends React.Component
{
      constructor(props)
      {
            super(props);
            this.state = {
            brand: "Ford ",
            model: "Mustang ",
```

```
            color: "red ",
            year: 1964
            };
      }
      changeColor = () => { this.setState({color: "blue "}); }
      render()
      {
            return (
            <div>
                  <h1>My {this.state.brand}</h1>
                  <p>
                        It is a {this.state.color}
                        {this.state.model}
                        from {this.state.year}.
                  </p >
                  <button type="button" onClick={this.changeColor}>Change color</button>
            </div>
            );
      }
}
export default Cars;
```

App.js

```
import './App.css';
import Cars from './Cars';

function App()
{
      return (
            <div className="App">
                  <Cars></Cars>
            </div>
      );
}

export default App;
```

Output:

My Ford

It is a red Mustang from 1964.

Change color

Change state

My Ford

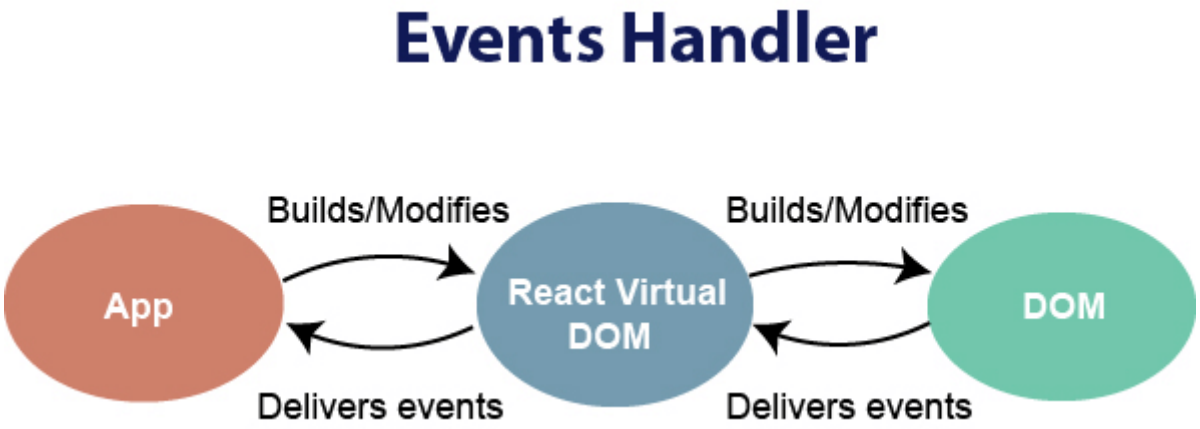
It is a blue Mustang from 1964.

Change color

2.4 EVENTS

An event is an action that could be triggered as a result of the user action or system-generated event. For example, a mouse click, loading of a web page, pressing a key, window resizes, and other interactions are called events.

React has its event handling system which is very similar to handling events on DOM elements. The react event handling system is known as Synthetic Events. The synthetic event is a cross-browser wrapper of the browser's native event.



Handling events with react has some syntactic differences from handling events on DOM. These are:

1. React events are named as **camelCase** instead of **lowercase**.
2. With JSX, a function is passed as the **event handler** instead of a **string**. For example:

Event declaration in plain HTML:

```
<button onclick="showMessage()">
    Hello world
</button>
```

Event declaration in React:

1. <button onClick={showMessage}>
2. Hello world
3. </button>

3. In react, we cannot return **false** to prevent the **default** behaviour. We must call **preventDefault** event explicitly to prevent the default behaviour. For example:

Example

Cars.js

```
import React from 'react';
class Cars extends React.Component
{
	constructor(props)
	{
		super(props);
		this.state = {
			brand: "Ford ",
			model: "Mustang ",
			color: "red ",
			year: 1964
		};
	}
	changeColor = () => {this.setState({color: "blue "});
}
render()
{
	return (
	<div>
		<h1>My {this.state.brand}</h1>
		<p>
			It is a {this.state.color}
			{this.state.model}
			from {this.state.year}.
		</p >
		<button type="button" onClick={this.changeColor}>Change color</button>
	</div>
    );
  }
}
export default Cars;
```

App.js

```
import './App.css';
import Cars from './Cars';
```

```
function App()
{
      return (
      <div className="App">
            <Cars></Cars>
      </div>
      );
}
export default App;
```

Output:

My Ford

It is a red Mustang from 1964.

Change color

Change state

My Ford

It is a blue Mustang from 1964.

Change color